ASTRONOMY IMAGE PROCESSING PIPELINE

Technical Details and Processing Design

ECE 379K • HST Team • Tamim Hasan, Humza Ahmad, Samuel Li

PURPOSE

This document consolidates the team's technical discussion into a reference for the end-to-end image-processing pipeline: CPU scheduling, GPU filtering and enhancement, source detection, validation, and output. The baseline system detects and measures image sources; it does not assign astrophysical object types.

1. System Scope and Expected Outputs

The pipeline accepts astronomical images in FITS format, reduces noise and imaging artifacts, detects source regions, measures them, and writes both a processed image and a source catalog. The CPU coordinates file I/O, region scheduling, reassembly, and storage. The GPU accelerates computationally intensive image filtering and enhancement.

1.1 Baseline responsibilities

- Read input FITS images and prepare image regions for processing.
- Estimate the image background and identify pixels that are statistically brighter than the local background.
- Group neighboring above-threshold pixels into candidate sources and reject candidates that are too small to be credible detections.
- Measure each source's centroid, bounding box, pixel count, and integrated brightness.
- Write the processed image and the per-source catalog to disk.

1.2 Explicit scope boundary

BASELINE LIMITATION

A detected source is not automatically labeled as a star, galaxy, asteroid, or another object type. The core output is a catalog of detected and measured sources, without a classification label.

Shape measurements and cross-matching against existing astronomical catalogs are possible extensions. Multi-band color analysis is not applicable to the current single-image processing path unless multi-filter data are added.

2. End-to-End Processing Flow

Stage	Primary responsibility	Key result
1. Ingest and inspect	CPU reads FITS data and evaluates regions of interest (ROIs).	Image data and candidate tiles
2. Schedule and batch	CPU prioritizes target regions, skips likely-empty tiles, and batches same-sized tiles.	Memory-safe GPU work batches
3. Filter and enhance	GPU applies transforms and selected denoising/enhancement operations.	Processed image tiles
4. Reassemble and detect	CPU reassembles overlaps, estimates background, groups pixels, and measures sources.	Source measurements
5. Store results	Output stage writes image files and catalog rows asynchronously.	Processed image plus source catalog

The stages should be pipelined rather than treated as a sequence of fully blocking steps. While the GPU processes one batch, the CPU can prepare another batch or handle results from earlier work, subject to available memory and queue capacity.

3. CPU Orchestration and GPU Scheduling

3.1 ROI selection and priority

The CPU-side dispatcher manages a work-item queue of ROI tiles. Tiles containing known target coordinates can receive higher priority than general survey tiles, so the most important regions are processed first when time or compute resources are limited.

- Coarse filtering: use a cheap thresholding pass or downsampled variance check to identify tiles that are likely to contain no useful signal. Skip these tiles before creating GPU batches.
- Uniform tile dimensions: group tiles of the same dimensions together, since batched FFT operations generally expect uniform input shapes. Example tile sizes in the design discussion are 512 × 512 and 1024 × 1024 pixels.

3.2 Batch formation and memory staging

- Batch capacity should be constrained by available GPU memory, including intermediate FFT buffers; do not choose batch size arbitrarily.
- Avoid excessive waiting to fill a batch. Dispatch when a memory-safe batch is full or a configured timeout/minimum-batch rule is reached.
- Stage transfer-bound input tiles in pre-allocated pinned (page-locked) host memory to improve host-to-device transfer performance.

3.3 Overlapping transfers and computation

Use multiple CUDA/HIP streams where supported so data transfer for a later batch can overlap GPU computation for an earlier batch. Alternating streams enables double- or triple-buffering instead of waiting for each transfer and compute operation to finish before starting the next.

3.4 Queueing and backpressure

Use bounded queues between pipeline stages. If reassembly, feature extraction, or disk output falls behind, the queue should limit how much additional data accumulates. This backpressure helps prevent unbounded memory growth and naturally slows upstream production until downstream stages catch up.

4. GPU Filtering and Enhancement

After a batch is transferred to the device, the GPU processes tiles through a set of numerical operations. The exact set and order of optional filters should be selected through validation. Intermediate data should remain on the GPU wherever practical to avoid repeated CPU–GPU transfers (“ping-pong”).

4.1 Forward FFT

A batched two-dimensional Fast Fourier Transform (FFT), using a library such as cuFFT or an equivalent implementation, converts each tile from the spatial domain to the frequency domain. In frequency space, certain filtering operations can be implemented as multiplications that would be more cumbersome in pixel space.

4.2 Frequency-domain filtering

Method	Role in the pipeline
Wiener filtering	Uses an estimate of signal-to-noise behavior to suppress noise while attempting to preserve useful detail.
Gaussian filtering	Applies a comparatively simple low-pass filter. It is computationally inexpensive, but can blur fine detail more aggressively.

These methods attenuate selected frequency components, often including high-frequency components associated with noise. They are alternatives or configurable choices; whether to use one or both should be established experimentally.

4.3 Spatial-domain denoising and artifact rejection

- Wavelet denoising: separates image structure by scale and suppresses noise-dominated components while attempting to retain real features.
- Bilateral filtering: smooths based on spatial proximity and intensity similarity, helping preserve sharp boundaries while reducing noise in relatively uniform regions.
- Cosmic-ray and hot-pixel rejection: identifies pixels that are anomalously bright compared with their neighbors. These artifacts can resemble isolated bright sources and may need dedicated handling.

Spatial-domain operations should be scheduled before the forward FFT or after the inverse FFT, as appropriate for the operation. Their placement depends on which domain the algorithm requires.

4.4 Image enhancement

- Contrast stretching remaps the image intensity range to make faint features easier to see while retaining bright-end detail.
- Unsharp masking sharpens fine structure by subtracting a blurred version of the image from the original.
- Point spread function (PSF) deconvolution, such as Richardson–Lucy, attempts to reverse telescope-induced blurring through iterative processing. It is computationally expensive and is a candidate to remove first if schedule or runtime constraints require a reduced pipeline.

4.5 Inverse FFT and return to the CPU

After the frequency-domain work is complete, the inverse FFT returns the data to the spatial domain. The resulting tiles are returned to host memory asynchronously. Overlap padding added during tiling must be trimmed or blended during reassembly to reduce visible seams and edge artifacts.

5. Source Detection and Measurement

The detection stage turns pixel data into a catalog. Its principal sequence is: estimate the background and noise, threshold pixels relative to that background, group adjacent qualifying pixels, filter small candidates, and measure each accepted source.

5.1 Brightness threshold

A fixed pixel-intensity cutoff is not robust when the sky background varies across an image. Instead, estimate a local background level and its noise, often represented by a standard deviation σ, using a sliding window or image tiles.

> **INITIAL THRESHOLD MODEL**
>
> Flag a pixel as above-background when: pixel intensity > local background + k × σ

A starting value for k is often in the range of approximately 3–5, but it must be tuned against the project's data. A lower threshold can recover fainter sources but may increase false positives; a higher threshold can suppress noise while missing dim sources.

5.2 Minimum source size

After grouping neighboring above-threshold pixels into candidate sources, apply a minimum connected-pixel count. The technical discussion suggests 3–5 pixels as an initial tunable range, not a universal constant. This filters isolated hot pixels and small random noise spikes that survive denoising.

The minimum-size rule complements, but does not replace, the separate cosmic-ray/hot-pixel rejection kernels. Detection settings should be tuned together because brightness and size criteria affect the balance between false positives and missed detections.

5.3 Catalog schema

Field	Meaning
source_id	Sequential identifier, unique within the catalog for one image.
centroid_x, centroid_y	Estimated source center in pixel coordinates. Floating-point values support sub-pixel precision.
bbox_x_min, bbox_y_min, bbox_x_max, bbox_y_max	Minimum and maximum x/y coordinates of the rectangle enclosing the source.
size_pixels	Number of pixels assigned to the source. This differs from bounding-box area because the source may not fill its rectangle.
integrated_brightness	Sum of source pixel intensities in raw image counts. It is not a calibrated astronomical magnitude and is primarily useful for relative comparisons within the same processing context.

Example CSV output (illustrative values):

```
source_id,centroid_x,centroid_y,bbox_x_min,bbox_y_min,bbox_x_max,bbox_y_max,size_pixels,integrated_brightness
1,142.37,89.12,138,85,147,94,42,15832.6
2,310.05,201.88,304,196,317,209,78,48201.3
3,55.91,477.63,52,474,60,482,27,6104.9
4,512.44,318.20,507,312,519,325,56,22017.8
```

6. CPU Post-Processing and Output

6.1 Reassemble the image

The CPU combines processed tiles into the full image. Because tiling uses overlap padding, the reassembly stage must trim or blend overlapping regions to reduce edge artifacts and visible seams.

6.2 Choose where feature extraction runs

Feature extraction can run on the fully reassembled image or on individual tiles, depending on image size and implementation trade-offs. Per-tile extraction can parallelize naturally across a CPU thread pool, but detections near tile boundaries must be deduplicated or merged so a single source is not reported more than once.

6.3 Optional catalog matching

If cross-matching is implemented, the CPU can compare detected source coordinates with existing astronomical catalogs. This is a lookup/matching task and does not generally require the GPU filtering stage.

6.4 Write results asynchronously

Artifact	Candidate formats	Contents
Processed image	FITS, PNG, or HDF5	The denoised and/or enhanced image.
Source catalog	CSV, FITS table, or Parquet	One row per detected source, including ID, centroid, bounding box, pixel count, and integrated brightness.

Buffered asynchronous writes prevent slow disk I/O from blocking all processing threads. While one batch is written, CPU workers should remain available to handle subsequent GPU results, within the limits imposed by bounded queues.

7. Validation and Threshold Tuning

Validation should use synthetic FITS images containing sources with known positions, sizes, and brightness values. This makes it possible to measure what the pipeline recovers instead of relying only on visual inspection.

1. Generate synthetic images with sources spanning a range of sizes and brightness levels, including faint and small cases that are likely to be difficult to detect.
2. Run the pipeline with candidate values for k, the minimum connected-pixel count, and other relevant processing settings.
3. Record whether each injected source is detected and compare recovered position, size, and brightness with the known values.
4. Track both missed sources (false negatives) and noise/artifact detections (false positives) to find a useful balance between completeness and reliability.

5. Compare GPU results numerically with the CPU reference and benchmark identical image batches.

> **PROJECT PERFORMANCE TARGET**
>
> The proposal defines success as processing the same image batches at least 2× as fast as the CPU-only version, without reducing measurement accuracy.

The proposal also identifies a planned real-image dataset of approximately 500 calibrated r-band FITS images from the Sloan Digital Sky Survey (about 6 GB uncompressed), alongside synthetic test images generated by the team.

8. Optional Classification and Reference Approaches

Object-type classification is outside the baseline scope. If it becomes a stretch goal, the following approaches are relevant to the project's existing detection and measurement architecture.

8.1 Shape-based separation

Stars usually appear as point sources whose measured profile is shaped by the telescope's point spread function (PSF), while galaxies are often spatially extended. One possible lightweight approach is to compare a source's full width at half maximum (FWHM), ellipticity, or related shape measures with the expected stellar PSF for the image. Optional measurements could include major_axis, minor_axis, and orientation_deg.

8.2 Source Extractor (SExtractor)

Source Extractor, also known as SExtractor, is a relevant reference implementation because its pipeline includes background estimation, thresholding, grouping, and source measurement. It also provides a neural-network-based star/galaxy classifier through its CLASS_STAR parameter. It can therefore serve as a comparison point for detection behavior and, only if classification is added, classification behavior.

8.3 Catalog cross-matching and multi-band data

- Catalog cross-matching: compare detections with existing catalogs that include object information; examples mentioned in the technical discussion are Gaia for stars and SDSS for galaxies.
- Multi-band photometry: use brightness measurements from multiple wavelength filters to add color information. This would require multi-band inputs and is not supported by the current single-image pipeline as described.

> **RECOMMENDED EXTENSION PATH**
>
> If classification is pursued later, shape-based analysis is the most direct first step because it builds on optional shape measurements and does not require new multi-band infrastructure or external catalog integration.

9. Key Parameters and Design Considerations

Parameter / decision	Why it matters	How to evaluate it
Background window or tile size	Controls how well local background and noise variations are captured.	Test on images with varying background levels.
Threshold multiplier k	Sets the sensitivity/noise trade-off.	Sweep candidate values and compare missed and false detections.
Minimum connected-pixel count	Rejects very small noise candidates but can discard small real sources.	Validate against synthetic sources of known size.
ROI tile dimensions and overlap	Affects FFT batching, memory use, edge artifacts, and tile-boundary detections.	Check seams and source recovery near tile borders.
GPU batch size and number of streams	Trades throughput against device-memory use and latency.	Benchmark with memory-safe batches and overlapping transfers.
Optional processing stages	Some operations, particularly iterative PSF deconvolution, may dominate runtime.	Compare accuracy and runtime; remove expensive optional steps first if necessary.

10. Glossary

Term	Definition in this project
FITS	Flexible Image Transport System, the standard astronomy image format used for input data.
ROI	Region of interest: an image region selected for scheduling or processing.
FFT / inverse FFT	Transforms between spatial and frequency representations of image data.
PSF	Point spread function: the image profile produced by an ideal point source through the imaging system.
Centroid	Intensity-weighted estimate of a source’s center, potentially at sub-pixel precision.
Pinned host memory	Page-locked CPU memory used to stage data for efficient host/device transfers.
CUDA/HIP stream	A device execution queue used to sequence and potentially overlap GPU operations.
Backpressure	A mechanism by which a slower downstream stage limits upstream production to control queue and memory growth.